



Факультет компьютерных наук
НИУ ВШЭ

08 октября 2026

Математическая модель мобильного робота и ROS2

Проектный семинар «Мобильные колесные роботы»

Дергачев Степан Алексеевич, sadergachev@hse.ru

Муравьев Кирилл Федорович, kmuravev@hse.ru



Математическая модель мобильного робота

- Модель описывает движение робота и позволяет прогнозировать его состояние, выбирать управление и оценивать перемещение.
- Вектор состояния робота $\mathbf{x}$ включает в себя набор величин, которые описывают его положение в пространстве и внутреннюю конфигурацию. Например: $\mathbf{x} = (p_x, p_y, \theta)$.
- Вектор управления $\mathbf{u}$ – это набор величин, которые подаются на исполнительные механизмы робота для изменения его состояния. Например: $\mathbf{u} = (v, w)$.

[TODO: Вид сверху:
мировые оси, оси
корпуса, точка (x, y) , угол
и система координат
датчика]



Математическая модель мобильного робота

- Для описания состояния робота (обычно) используется неподвижная мировая система координат.
- Выделяют систему координат корпуса робота, которая движется вместе с роботом.
- Составные части робота (в том числе датчики) могут обладать своими системами координат.

[TODO: Вид сверху:
мировые оси, оси
корпуса, точка (x, y) , угол
и система координат
датчика]



Прямая и обратная задача кинематики

Прямая задача кинематики

- По управлению определить изменение состояния робота.

[TODO: Схема]

Обратная задача кинематики

- По требуемой изменению состояния и времени копределить необходимые управления.

[TODO: Схема]



Модель мобильного робота как динамическая система

Непрерывная модель

- Состояние и управление определены в каждый момент времени. Модель задает изменение состояния:

$$\dot{\mathbf{x}} = f(\mathbf{x}(t), \mathbf{u}(t)),$$

Дискретная модель

- Состояние рассматривается в отдельные моменты времени t . Модель задает переход от состояния к состоянию:

$$\mathbf{x}_{t+1} = F(\mathbf{x}_t, \mathbf{u}_t),$$

Модель робота также может включать: модель выхода системы $\mathbf{y}$ (например, модель показаний датчиков в зависимости от состояния и управления), ограничения на состояние и управление, неточности параметров и случайные возмущения.



Ограничения

Тип	Что ограничивают
Геометрические	Допустимые положения элементов системы
Кинематические	Допустимые скорости движения элементов системы
Динамические	Ограничения возможных ускорений системы, сил, моментов приводов и т.д.

Примеры явных ограничений

- $\|v\| \leq v_{\max}, \|w\| \leq w_{\max}$ – ограничения на максимальные скорости
- $\|\dot{v}\| \leq a_{\max}$ – ограничение на максимальное ускорение



Неопределенность в модели

Шум изменения состояния

$$\mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k) + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, \Sigma_k)$$

Описывает ошибку шага модели. Позволяет учитывать разные причины, но параметры шума сложнее связать с физикой движения.

Шум управления

$$\mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k + \varepsilon_k), \quad \varepsilon_k \sim \mathcal{N}(0, \Sigma_k)$$

Описывает ошибки исполнения управления. Их влияние на позу зависит от кинематики.

* Нормальное распределение – допущение модели, могут использоваться другие распределения.



Виды колесных платформ

[TODO: нужны картинки]

Дифференциальный привод

Независимое вращение левого и правого колес. Поворот за счет разности скоростей.

Omni-колеса

Свободные ролики на колесе и несколько направлений установки колес позволяют двигаться боком.

Автомобильная платформа

Поворот за счет руления. Есть ограничение на минимальный радиус поворота.

Колеса Mecanum

Наклонные ролики на колесе. При согласованном вращении колес платформа может двигаться боком.



Модель Single Integrator

- Состояние – позиция $\mathbf{x} = (p_x, p_y)$.
- Управление – скорость в мировой системе координат: $\mathbf{u} = (v_x, v_y)$.
- Направление скорости может меняться мгновенно.

[TODO: Картинка с кружочком и скоростью]

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{u}_k \Delta t \quad \|\mathbf{u}\| \leq v_{\max}$$

Удобная абстракция для планирования, но не учитывает многих ограничений колесной платформы (ускорение, невозможность движения "боком" и т.д.).



Модель Double Integrator

- Состояние – положение и скорость:
 $\mathbf{x} = (p_x, p_y, v_x, v_y)$.
- Управление – ускорение $\mathbf{u} = (a_x, a_y)$ в мировой системе координат.

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{u}_k \Delta t^2,$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{u}_k \Delta t$$

$$\|\mathbf{u}_k\| \leq a_{\max}, \quad \|\mathbf{v}_k\| \leq v_{\max}$$

[TODO: Картинка с кружочком и скоростью]

Учитывает ограничение на ускорение, но не описывает невозможность движение "боком".



Модель Differential-drive

- Два независимо приводимых в движение колеса радиуса r . Расстояние между колесами L .
 - Состояние робота задается позицией середины оси колес и направлением $\mathbf{x} = (p_x, p_y, \theta)$.
 - Предполагается, что колеса движутся без продольного и бокового проскальзывания. Боковая скорость всегда равна нулю.
 - Движение через изменения угловой скорости колес w_L, w_R . Можно вычислить продольные скорости v_L, v_R центров колес :
- $w_L = w_R$: прямая.
 - $w_L \neq w_R$: дуга.
 - $w_L = -w_R \neq 0$: вращение на месте.

[TODO: картинка со схемой робота]



Модель Differential-drive

- Удобнее использовать для управления линейную и угловую скорости корпуса:

$$v = \frac{(v_R + v_L)}{2}, \quad w = \frac{(v_R - v_L)}{L}$$

- Непрерывная модель:

$$\begin{cases} \dot{x} = v \cos \theta, \\ \dot{y} = v \sin \theta, \\ \dot{\theta} = w \end{cases}$$

- $w_L = w_R$: прямая.
- $w_L \neq w_R$: дуга.
- $w_L = -w_R \neq 0$: вращение на месте.

[TODO: картинка со схемой робота]



Модель Differential-drive

Дискретная модель: метод Эйлера

$$\begin{cases} p_{x,k+1} = p_{x,k} + v_k \cos \theta_k \Delta t, \\ p_{y,k+1} = p_{y,k} + v_k \sin \theta_k \Delta t, \\ \theta_{k+1} = \theta_k + w_k \Delta t \end{cases}$$

Дискретная модель: мгновенный центр вращения

Более точное вычисление – через мгновенный центр вращения (ICC):

$$R = \frac{v}{w} = \frac{L}{2} \cdot \frac{w_R + w_L}{w_R - w_L}$$

$$(ICC_x, ICC_y) = (p_x - R \sin \theta, p_y + R \cos \theta)$$

[TODO: картинка с ICC]



Оценка перемещения: Одометрия

Одометрия – оценка перемещения робота на основе данных о его движении

- Накопление приращений дает позу относительно начала движения
- Одометрия $\neq$ локализация.

Пример	Источник
Одометрия по управляющим сигналам	Заданные скорости и модель движения.
Колесная одометрия	Углы поворота или скорости колес по энкодерам.
Инерциальная одометрия	Ускорения и угловые скорости по IMU.
Визуальная одометрия	Изменение кадров в последовательности изображений с камеры.
Лидарная одометрия	Изменение лучей в последовательности сканов LiDAR.



Ошибки одометрии

- Проскальзывание колес, неточно заданные параметры робота, шум датчиков приводят к небольшим ошибкам на каждом шаге.
- Небольшие ошибки со временем накапливаются.

Источник	Ограничения
Управление	Точное исполнение команд невозможно, никак не учитываются проскальзывания и застревания робота
Энкодеры	Измеряют вращение колес, но не обнаруживают проскальзывания.
IMU	Погрешности измерений в угловых скоростях и ускорениях очень быстро накапливаются при интегрировании.
Камеры	Сильная зависимость от текстуры, освещения и движения объектов в среде, вычислительная сложность
LiDAR	Сильная зависимость от наполнения среды, вычислительная сложность



Системы координат в ROS2

- `tf2` – библиотека в ROS2, предназначенная для отслеживания, вычисления и хранения преобразований между системами координат в виде дерева.
- Лист дерева (frame) – конкретная система координат.
- Ребро дерева (transform) – перенос и поворот дочерней системы относительно родительской.
- Направление осей: OX вперед, OY влево, OZ вверх (правая система).
- Единицы измерения: метры и радианы.
- Поворот хранится кватернионами, но может задаваться через углы Эйлера.

[TODO: схема дерева]



Системы координат в ROS2

Конвенция имен

- `base_link` – система координат корпуса робота.
- `base_footprint` – система координат проекцию корпуса на пол.
- `odom` – система координат точки начала движения.
- `map` – система координат карты.
- Системы координаты колес и датчиков дочерние к `base_link`.
- Одометрия задает `odom` $\rightarrow$ `base_link`, локализация задает `map` $\rightarrow$ `odom`.

[TODO: схема дерева]



Системы координат в ROS2

- Передача информации о преобразованиях происходит через публикации в специальных топиках
- Преобразования делятся на статические (топик `/tf_static`) и динамические с временными метками (топик `/tf`).
- Объект `Buffer` хранит преобразования, `TransformListener` получает, `TransformBroadcaster` публикует.
- Методы `lookup_transform` и `transform` используются для преобразований.

```
1 from tf2_ros import Buffer, TransformListener
2 from rclpy.time import Time
3 import tf2_geometry_msgs
4 ...
5 self.tf_buffer = Buffer()
6 self.tf_listener = TransformListener(self.tf_buffer, self)
7 tf = self.tf_buffer.lookup_transform('base_link', 'laser', Time())
8 point_base = self.tf_buffer.transform(point, 'base_link')
```



Управление роботом в ROS2

Аппаратный интерфейс		Интерфейс взаимодействия с компонентами робота или роботом в симуляции
ros2_control	controller_manager	Управление состоянием контроллеров
	diff_drive_controller	Преобразование скоростей корпуса в скорости колес, вычисление колесной одометрии
	joint_state_broadcaster	Чтение состояния суставов робота из аппаратного интерфейса и публикация в формате joint_states
robot_state_publisher		Публикация преобразований между компонентами робота в /tf и /tf_static на основе joint_states и описания в специальном формате URDF



Управление роботом в ROS2

[TODO: Единая схема]



Одометрия в ROS2

- Одометрия публикуется в двух вариантах:
- В топике `/odom` в формате `nav_msgs/msg/Odometry`: позиция, скорость и ковариации.
- В преобразовании `odom` → `base_link`: только позиция, без скоростей и ковариаций.
- Поза и время должны быть согласованы. У преобразования должен быть один издатель.

```
1 ros2 topic echo /odom
2 ros2 run tf2_ros tf2_echo odom base_link
```



Визуализация и диагностика

- `tf2_echo` – посмотреть отдельное преобразование.
- `tf2_tools view_frames` – дерево преобразований систем координат,
- `RViz2` – визуализация модели робота, TF, карты, данных датчиков и д.р. в 3D
- `Foxglove` – 3D-сцена, графики, просмотр сообщений, просмотр дерева преобразований и многое другое.

```
1 ros2 run tf2_ros tf2_echo odom base_link
2 ros2 run tf2_tools view_frames
3 ros2 run rviz2 rviz2
4 ros2 launch foxglove_bridge foxglove_bridge_launch.xml
```



Спасибо за внимание!